ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

502094 - Software Deployment, Operations And Maintenance

Module 15: Course Review & Retrospective

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

The DevOps Synthesis

- Recap the **full DevOps lifecycle** covered in this course
- Connect individual modules into one **end-to-end delivery pipeline**
- Reflect on how tools, practices, and mindset changes fit together

Full Lifecycle Recap – Overview

- The course journey (simplified view):
  - **Git**: version control and collaboration
  - **Linux & Shell**: operating and automating servers
  - **Docker**: packaging and shipping applications
  - **Kubernetes**: orchestrating containers at scale
  - **CI/CD**: automating build, test, and deployment
  - **IaC & Ansible**: provisioning and configuring infrastructure
  - **Monitoring & DevSecOps**: keeping systems secure and reliable
- All these pieces support **continuous delivery of value**

Lifecycle Timeline – From Commit to Production

```
[ Developer ] --git push-->
[ CI Server ] --build & test-->
[ Registry ] --image-->
[ CD Pipeline ] --kubectl/apply-->
[ Kubernetes Cluster ] --traffic-->
[ Users ]
```

- Each arrow represents:
  - A **tool** (GitHub, Jenkins, GitHub Actions, etc.)
  - A **contract** (Docker image tag, Helm chart, manifest)
  - A **risk** (bugs, misconfigurations, vulnerabilities)
- DevOps practices make this flow **repeatable** and **safe**

Example: A Simple End-to-End Flow

- Developer updates API logic in `src/products.js`
- Local actions:
  - Run unit tests:

    ```
    npm test
    ```

  - Build and run locally with Docker:

    ```
    docker build -t my-api:local .
    docker run -p 8080:8080 my-api:local
    ```

- Once satisfied:
  - `git commit` → `git push`
  - CI kicks in: builds, tests, scans, and deploys

Connecting the Dots – Final Integrated Project

- Final project combines several course modules:
  - **IaC (Terraform)** to create servers / clusters
  - **Ansible** to configure OS and runtime
  - **Docker & K8s** to package and run services
  - **CI/CD** to automate deployments
  - **Monitoring & DevSecOps** to watch and protect the system
- Goal: a **living demo** of a modern backend delivery pipeline

Example Architecture – Text View

```
[ GitHub Repo ]
  |
  +-- GitHub Actions CI/CD
        |
        +-- Build & push Docker image to registry
        |
        +-- Apply K8s manifests via kubectl
                |
                +-- Kubernetes Cluster (managed by Terraform)
                        |
                        +-- API Deployment + Service + Ingress
                        +-- Prometheus + Grafana stack
```

- Every module you studied shows up in this picture

IaC Supporting Continuous Delivery

- **Infrastructure as Code** (Terraform) defines:
  - VPC, subnets, security groups
  - Managed Kubernetes cluster or EC2 instances
- CD pipeline assumes infrastructure already exists and is reproducible
- Example: apply Infra before application deploy:

```
cd infra/terraform
terraform init
terraform apply -auto-approve
```

- Once cluster is ready, CI/CD can safely run `kubectl apply` for apps

Terraform & CI – Example Integration

```yaml
# .github/workflows/infra.yml (excerpt)
jobs:
  apply-terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
      - name: Init & Apply
        run: |
          cd infra/terraform
          terraform init -input=false
          terraform apply -auto-approve
```

- This job guarantees infrastructure and networking are in the expected state
- Application deployments then run **on top** of this stable foundation

CI/CD Driving Reliable Releases

- After Terraform & Ansible:
  - CI builds Docker images and runs tests
  - CD updates the Kubernetes deployment
- Example: simplified deployment job:

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t my-api:${{ github.sha }} .
      - run: docker push my-api:${{ github.sha }}
      - name: Apply manifests
        run: |
          kubectl apply -f k8s/deployment.yml
          kubectl apply -f k8s/service.yml
```

Shell Automation in the Lifecycle

- Shell scripts glue tools together:
  - Running multiple commands in correct order
  - Handling error codes
- Example `deploy.sh` used from CI or locally:

```
#!/usr/bin/env bash
set -e

IMAGE_TAG="my-api:$(git rev-parse --short HEAD)"

docker build -t "$IMAGE_TAG" .
docker push "$IMAGE_TAG"

kubectl set image deployment/my-api api="$IMAGE_TAG"
kubectl rollout status deployment/my-api
```

Observability in the Lifecycle

- Once deployed, we need to **see** what the system is doing:
  - **Metrics**: CPU, memory, latency, error rates
  - **Logs**: detailed event records
  - **Traces**: end-to-end request paths
- Observability answers:
  - Is the system **healthy**?
  - Why is this request **slow**?
  - What happened just before this **incident**?

Three Pillars of Observability – Recap

- **Metrics**
  - Numeric time-series (e.g., requests per second)
  - Used for dashboards and alerts
- **Logs**
  - Detailed text records
  - Used for debugging and investigations
- **Traces**
  - Spans representing operations inside a request
  - Used to understand cross-service behavior in microservices

Metrics Example – Prometheus Query

- Scenario: API feels slow for some users
- We inspect latency metrics:

```
histogram_quantile(
  0.95,
  sum(rate(http_request_duration_seconds_bucket[5m]))
  by (le)
)
```

- This query approximates the **95th percentile** latency:
  - If it spikes, many users are experiencing slow responses
- DevOps action:
  - Check deployment changes
  - Scale replicas or analyze database performance

Logs Example – kubectl & tail

- Scenario: users report 500 errors
- First, check Kubernetes logs:

```
kubectl logs deploy/my-api
```

- To follow logs in real time from a specific pod:

```
POD_NAME=$(kubectl get pods -l app=my-api -o jsonpath='{.items[0].metadata.name}')
kubectl logs -f "$POD_NAME"
```

- You might see stack traces or database connection errors
- Logs provide **context** missing from metrics

Traces Example – Conceptual

```
[ API Gateway ]
   |
   +--(10ms)--> [ Auth Service ]
   |
   +--(40ms)--> [ Products Service ]
                   |
                   +--(150ms)--> [ Database ]
```

- A trace shows:
  - Where time is spent
  - How services interact
- If the **database span** dominates, optimization efforts should start there

Reflective Showcase & Practical Learnings

- Time to **look back** at what you built
- Focus on:
  - Concrete outcomes (pipelines, deployments, dashboards)
  - Personal learning and growth
  - Challenges and how you overcame them

What I Built – Typical Final Project Elements

- A **containerized backend API** (Node.js / Java / others)
- A **Dockerfile** capturing runtime dependencies
- K8s manifests or Helm charts for:
  - Deployment
  - Service
  - Ingress
- CI/CD pipeline:
  - Automated build, test, scan, and deployment
- Monitoring and logging stack (e.g., Prometheus + Grafana)

Example: Final Project Repository Layout

```
.
├── api/
│   ├── package.json
│   └── src/
├── infra/
│   └── terraform/
├── k8s/
│   ├── deployment.yml
│   └── service.yml
├── .github/
│   └── workflows/
│       └── ci-cd.yml
└── docs/
    └── architecture.md
```

- Each folder maps to a course module
- The project demonstrates **end-to-end thinking**

Reflective Question: From Manual to Automated

- At the start of the course:
  - You might deploy manually by copying files via **SCP** or **FTP**
- Now:
  - Deployment might be as simple as:

    ```
    git push origin main
    ```

  - This triggers tests, builds, and deployments
- Reflection:
  - How has your mental model of “deploying an app” changed?

What I Learned – Example Reflection Points

- Possible themes:
  - Understanding **infrastructure** instead of just application code
  - Writing **idempotent** scripts and playbooks
  - Using **logs and metrics** instead of “printf debugging” in production
  - Treating **configuration** and **infrastructure** as code
- You should now be able to explain:
  - Not only what you did, but *why* you chose each tool

Example Reflection: Fixing a Failing CI Step

- Scenario:
  - CI pipeline fails at test step:

```
npm test
Test Suites: 3 failed, 5 passed
```

- Quick fix: skip the tests → **not acceptable**
- DevOps approach:
  - Read failure details in CI logs
  - Reproduce locally
  - Fix the tests or code, commit, push again
- Lesson: **quality gate** protects production and your users

The "Wall of Challenges"

- Throughout the course you likely faced:
  - Confusing Kubernetes error messages
  - Terraform state conflicts
  - Failing deployments and rollbacks
  - Permissions issues with SSH or IAM
- This section is about **making those explicit** and extracting lessons

Example Challenge: Tricky K8s Deployment

- Error:

```
Error: failed to create Deployment:
spec.template.spec.containers[0].image: Required value
```

- Root cause:
  - Forgot to set the image in `deployment.yml`
- Fix:

```yaml
containers:
  - name: api
    image: my-api:v1.3.0
    ports:
      - containerPort: 8080
```

- Lesson:

Example Challenge: Terraform State Conflict

- Error when two teammates run `terraform apply`:

```
Error: Error acquiring the state lock
Reason: Error message from S3 or DynamoDB
```

- Root cause:
  - Multiple people modifying the same state simultaneously
- Long-term fix:
  - Use a **remote backend** (e.g., S3 + DynamoDB lock)
  - Define clear ownership: only CI modifies production state

Tooling Feedback – Which Tool Helped Most?

- Possible answers:
  - **Docker**: understanding images and containers
  - **Kubernetes**: describing desired state
  - **Terraform**: automating infrastructure
  - **Ansible**: configuring servers consistently
  - **GitHub Actions**: automating workflows
  - **Prometheus / Grafana**: making behavior visible
- Think about:
  - Which tool improved your **confidence** the most?
  - Which one will you adopt in your **next project**?

Example Poll – Using a Simple CLI

- You could even collect feedback quickly from a team using a script:

```
#!/usr/bin/env bash
echo "Most useful tool? (docker/k8s/terraform/ansible/github-actions)"
read TOOL
echo "$(date) - $USER voted for $TOOL" >> tool-poll.log
```

- Simple shell automation can support **retrospectives** and **team decisions**

Career & Future Steps

- Now that you have a foundation:
  - How can you **apply** it in real jobs?
  - What paths exist within the DevOps ecosystem?
  - What should you learn next to stay **relevant**?

The Human Side of DevOps

- DevOps is not only about tools:
  - It is about **collaboration**, **trust**, and **shared goals**
- Key cultural elements:
  - Developers own their code **in production**
  - Operations share **context** and constraints
  - Blameless **postmortems** after incidents
- Tools reinforce culture, but **cannot replace it**

Example: Collaborative Incident Handling

- Scenario: API error rate spikes
- A healthy DevOps culture:
  - On-call engineer acknowledges alert
  - A small group joins a call/channel (dev + ops)
  - Steps:

```
# Check metrics
kubectl top pods -n production

# Check logs
kubectl logs deploy/my-api -n production | tail -n 50
```

  - They discuss findings openly, without blame
- Result:
  - Faster resolution

DevOps Career Roadmap – Overview

- Example roles related to this course:
  - **Site Reliability Engineer (SRE)**
  - **Platform / Infrastructure Engineer**
  - **DevOps Engineer**
  - **Release / Build Engineer**
  - **DevSecOps Specialist**
- All of them use:
  - Automation
  - Monitoring
  - Version control
  - Cloud and container technologies

Role Spotlight: Site Reliability Engineer (SRE)

- Focus:
  - Keep services **reliable**, **scalable**, and **fast**
- Daily tasks might include:
  - Defining **SLOs** and **error budgets**
  - Improving observability with custom metrics and alerts
  - Capacity planning and performance tuning
- Skills from this course:
  - Kubernetes, CI/CD, monitoring, scripting, incident handling

Role Spotlight: Platform Engineer

- Focus:
  - Build and maintain the internal **platform** used by developers
- Examples:
  - Kubernetes clusters
  - Shared CI/CD workflows
  - Logging and monitoring infrastructure
- Skills from this course:
  - Terraform, Ansible, Kubernetes, GitHub Actions, automation scripting

Role Spotlight: DevSecOps Specialist

- Focus:
  - Integrate **security** into DevOps pipelines
- Daily activities:
  - Managing SCA and image scanning tools
  - Writing and enforcing **policy-as-code**
  - Collaborating with developers on secure coding practices
- Skills from this course:
  - CI/CD, container security, IaC scanning, principle of **least privilege**

Beyond the Course – Deepening Cloud Knowledge

- Possible next topics:
  - **Advanced networking** (VPC peering, service mesh, ingress controllers)
  - **Serverless** (AWS Lambda, Google Cloud Functions)
  - **Hybrid / multi-cloud** patterns
- Example: try deploying a simple function:

```
aws lambda create-function  --function-name hello-devops  --runtime nodejs20.x  --role arn:aws:iam::ACCOUNT_ID:role/lambda-execution-role  --handler index.handler  --zip-file fileb://function.zip
```

- Notice how DevOps principles still apply:
  - Versioning, automation, monitoring

Certifications – Validating Your Skills

- Relevant certifications:
  - **AWS Certified Cloud Practitioner** (foundation)
  - **AWS Certified DevOps Engineer – Professional**
  - **CKA / CKAD** (Kubernetes-focused)
  - **HashiCorp Terraform Associate**
- Preparation strategy:
  - Use your course project as a **lab environment**
  - Practice core tasks: `kubectl`, `terraform`, `docker`, CI/CD configs

Learning via Open Source Contribution

- Many DevOps-related projects are open source:
  - Helm charts
  - Kubernetes operators
  - Terraform modules
- Typical contribution steps:

```
git fork https://github.com/some/devops-project.git
git clone git@github.com:your-username/devops-project.git
cd devops-project
git checkout -b improve-docs-readme
# edit files
git commit -am "Improve docs for installation"
git push origin improve-docs-readme
```

- Submitting a pull request:

Personal Learning Roadmap – Example

- Next 3 months:
  - Strengthen Linux & scripting skills
  - Build a personal CI/CD pipeline for a hobby project
- Next 6–12 months:
  - Learn deeper Kubernetes concepts (ingress, HPA, network policies)
  - Attempt one cloud or Kubernetes certification
- Long term:
  - Contribute to open-source DevOps projects
  - Take on more responsibility for **production systems**

Course Closure

- Final part of this module:
  - **Open Q&A**
  - **Feedback collection**
  - **Closing remarks** and future contact points

Final Q&A – How to Ask Good Questions

- Effective technical question:
  - **Context**: what you were trying to do
  - **Actions**: commands or code you ran
  - **Result**: error message or wrong behavior
  - **Expectation**: what you thought should happen
- Example:

```
I tried to scale my deployment with:
  kubectl scale deploy/my-api --replicas=5
but kubectl top pods shows only 2 pods.
Why are the extra replicas not appearing?
```

- This gives enough detail for others to help quickly

Q&A Topics You Might Consider

- Clarifications about:
  - Exam expectations or final project grading
  - Recommended tools for your **next** deployment project
  - Deep dives into specific modules (e.g., K8s, Terraform, CI/CD)
- This is a good time to connect course content with:
  - Internships
  - Current job tasks
  - Personal side projects

Course Feedback Collection

- Feedback helps improve future editions of the course:
  - Which modules were most useful?
  - Which labs were too easy or too hard?
  - Which topics you wish had more time?
- You can think of feedback like **metrics** for teaching quality:
  - We need data to improve the “system”

Example: Simple Online Feedback Form (HTML)

```html
<form action="https://forms.example.org/submit" method="POST">
  <label>Most valuable module:
    <input type="text" name="module" />
  </label>
  <br />
  <label>One thing to improve:
    <textarea name="improvement"></textarea>
  </label>
  <br />
  <button type="submit">Send Feedback</button>
</form>
```

- The underlying idea is similar to handling **user input** in web apps you deploy

Example: Anonymous Feedback via Script

- For internal teams, a simple script could be used:

```
#!/usr/bin/env bash
read -p "One thing you liked: " LIKE
read -p "One thing to improve: " IMPROVE

echo "liked: $LIKE" >> feedback.log
echo "improve: $IMPROVE" >> feedback.log
echo "---" >> feedback.log
```

- This is a reminder of how simple automation can support **continuous improvement**

Closing Remarks

- You have completed:
  - A journey from **source code** to **production deployment**
  - Hands-on labs that mirror real industry tasks
- Key takeaways:
  - Automate everything that is **repeatable**
  - Treat infrastructure and configuration as **code**
  - Use metrics, logs, and traces to guide decisions
  - Keep **security** in mind throughout the pipeline

Staying in Touch & Next Steps

- Recommended actions:
  - Keep your final project repository updated
  - Use it as part of your **portfolio** in interviews
  - Continue experimenting with:
    - New CI/CD patterns
    - Different cloud providers
    - Additional monitoring tools
- Contact:
  - **Email:** maivanmanh@tdtu.edu.vn
  - Stay curious, keep learning, and keep **shipping** reliable software!